

Agentic AI Knowledge Platform

Architecture & System Design

A prototype internal knowledge assistant combining Retrieval-Augmented Generation (RAG), a knowledge graph (GraphRAG), and an agentic router behind a FastAPI backend. Designed to run fully self-contained with zero API keys, and to scale to managed cloud services (OpenAI, Neo4j, Astra DB, Redis) by configuration alone.

Prepared for	AI/ML Engineer Take-Home Assessment — AWTG
Author	Asif
Deliverable	8.3 Architecture Diagram + 8.4 Data Model + SYSTEM_DESIGN
Runtime posture	Self-contained / offline-first, key-optional (LiteLLM)

1. Problem & Design Goals

MedNova Solutions is a healthcare-technology company whose internal knowledge is scattered across project briefs, architecture notes, meeting summaries, risk assessments and requirements. Employees waste time searching manually and re-asking the same questions. The platform lets an employee ask a natural-language question and receive a **reliable, source-backed answer** — not a generic chatbot reply, but one grounded in the company's own documents and the relationships between projects, technologies, risks and requirements.

Design goals that shaped every decision below:

- **Grounded & honest.** Every answer cites source documents and the system explicitly says when the documents do not contain the answer.
- **Runs anywhere in minutes.** Zero mandatory API keys or cloud accounts; local embeddings, an in-process vector index and an in-memory graph mean a reviewer can clone and run immediately.
- **Swap-by-config to production.** Each layer hides behind an interface so OpenAI, Neo4j, Astra DB and Redis can replace the local defaults without code changes.
- **Relationships are first-class.** A knowledge graph answers connection questions ("what is the relationship between the platform, Neo4j and Astra DB?") that pure vector search handles poorly.

2. System Architecture

A request enters the FastAPI backend, which validates it and checks the cache. On a miss, the **agentic router** classifies the question and dispatches it to the RAG pipeline, the GraphRAG pipeline, a summarisation route, or a combination. Retrieved context is passed to the LLM provider layer, which uses a real LLM when a key is configured and otherwise falls back to a deterministic, extractive answerer that composes a grounded answer straight from the retrieved chunks. Ingestion runs as a background task so large uploads never block the API.

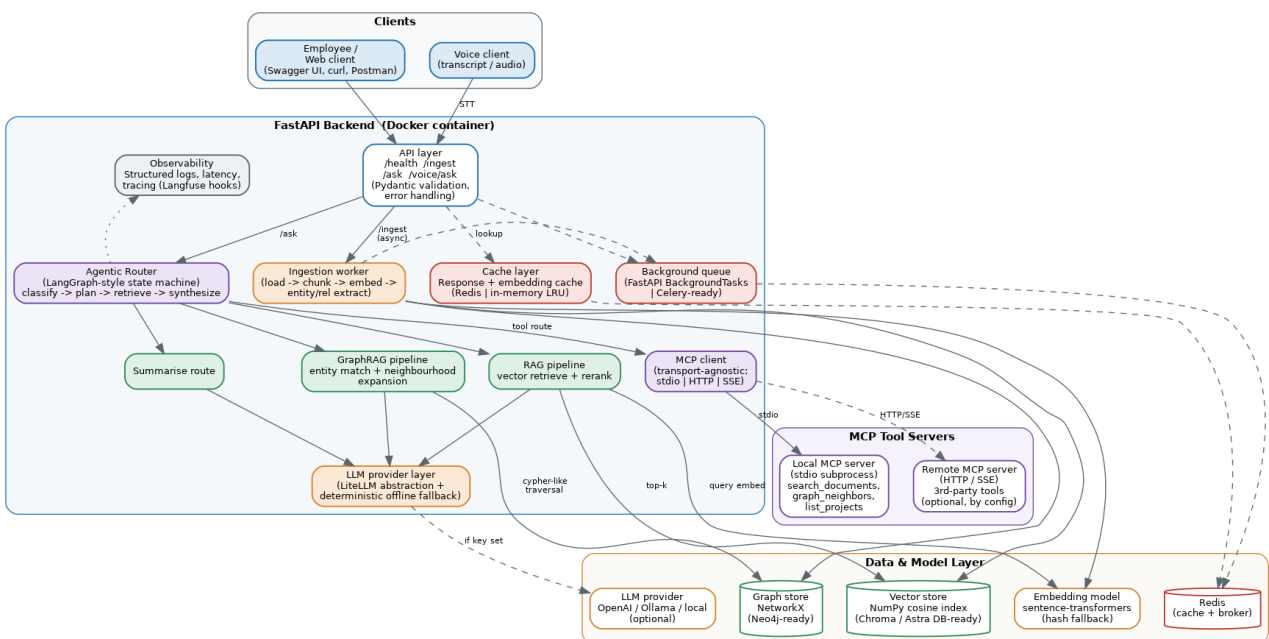


Figure 1 — End-to-end architecture. Dashed edges are optional / config-gated (Redis, managed LLM). Cylinders are data stores.

3. Component Responsibilities

API layer	FastAPI. Endpoints /health, /ingest, /ask, /voice/ask. Pydantic request/response models, input validation, structured error handling, per-request id.
Agentic router	LangGraph-style state machine: classify -> plan -> retrieve -> synthesize -> verify. Rule + heuristic classifier selects vector, graph, hybrid or summary route; falls back to hybrid when uncertain.
RAG pipeline	Embeds the query, runs cosine top-k over the vector store, applies a light lexical rerank, and builds a grounded, citation-carrying prompt.
GraphRAG pipeline	Matches entities in the question, expands their neighbourhood in the graph (1–2 hops), and returns connected projects / tech / risks plus the supporting documents. Combined with vector hits for hybrid questions.
LLM provider layer	LiteLLM-style abstraction. One interface, many providers (OpenAI, Ollama, local). Deterministic extractive fallback guarantees offline operation and reduces hallucination.
Ingestion worker	Loads .md/.txt/.pdf/.docx, chunks with overlap, embeds, and extracts entities + relationships into the graph. Runs via the background queue.
Vector store	NumPy cosine index persisted to disk. Interface mirrors Chroma / Astra DB so it can be swapped with no caller changes.
Graph store	NetworkX multi-digraph persisted to JSON; Cypher-like helpers. Neo4j-ready via the same GraphStore interface.
Cache layer	Response + embedding cache. Redis when REDIS_URL is set, otherwise an in-memory TTL-LRU. Keyed by sha256(question).
Queue / background	FastAPI BackgroundTasks for async ingestion; Celery-ready design documented for horizontal scaling.
Observability	Structured JSON logs, per-request latency, route + retrieval traces, optional Langfuse tracing when keys are present.
MCP client	Transport-agnostic Model Context Protocol client. Talks to a local studio tool server today and remote HTTP/SSE servers by config only. Lets the agent invoke external tools; degrades gracefully if absent.

4. Agentic Workflow & Routing

The router is the heart of the "agentic" behaviour: it decides *how* to answer rather than blindly running one pipeline. Classification uses cheap heuristics first (relationship verbs, entity co-occurrence, summary intent) and can escalate to an LLM classifier when a key is available.

Vector route	Fact-lookup / "which documents mention X" — semantic top-k retrieval.
Graph route	Relationship questions — "relationship between A, B and C", "what is connected to the patient assistant".
Hybrid route	"Which projects use both RAG and GraphRAG" — graph narrows candidates, vector supplies evidence ser
Summary route	"Summarise the architecture" — broad multi-chunk retrieval + summarise prompt.

Voice route	/voice/ask — transcript (or STT) -> same router -> optional TTS.
Tool route	Invokes an external tool over MCP (e.g. live data). Local stdio server now; remote HTTP/SSE servers by co

Every route ends in a **verify** step: if retrieval confidence is below threshold, the system returns an explicit "not enough information in the documents" answer instead of guessing.

External tools via MCP. The agent can also call tools through a transport-agnostic **Model Context Protocol** client. A local MCP server (spawned over stdio) exposes the platform's own retrieval as standard tools (*search_documents*, *graph_neighbors*, *list_projects*) plus generic tools; switching to a **remote** MCP server is a pure config change (stdio → HTTP/SSE), so third-party or shared tool servers plug in without code changes. If the MCP layer is unavailable the tool route degrades to vector search.

5. Data & Knowledge Model

Two complementary stores. The **vector store** holds chunk embeddings for semantic recall. The **knowledge graph** holds typed entities and relationships for reasoning over connections. Both are linked back to the source document so every answer is traceable.

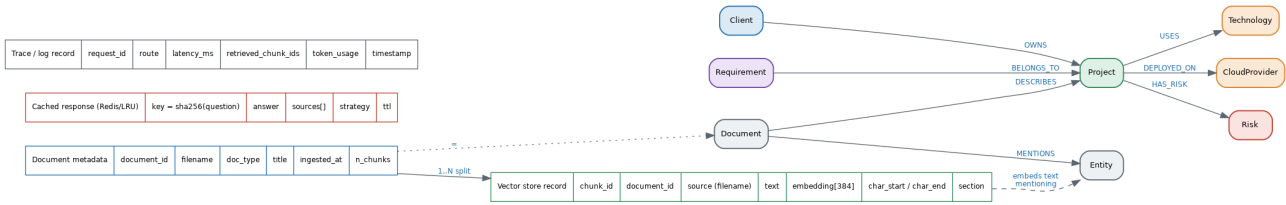


Figure 2 — Data model. Left: knowledge-graph node/relationship types. Right: vector-store, document, cache and trace records.

Graph schema (Cypher-style):

```
(:Client)-[:OWNS]->(:Project)
(:Project)-[:USES]->(:Technology)
(:Project)-[:DEPLOYED_ON]->(:CloudProvider)
(:Project)-[:HAS_RISK]->(:Risk)
(:Requirement)-[:BELONGS_TO]->(:Project)
(:Document)-[:MENTIONS]->(:Entity)
(:Document)-[:DESCRIBES]->(:Project)
```

6. Caching, Queues, Observability & Scale

Caching. Two layers. (1) A response cache keyed on the normalised question hash returns identical answers instantly and cuts LLM cost. (2) An embedding cache avoids re-embedding repeated text during ingestion and query. Backend is Redis in production and an in-memory TTL-LRU locally — same interface. A note on **KV / prompt cache**: for a hosted LLM we additionally benefit from provider-side prompt caching by keeping the system prompt and retrieved context prefix-stable.

Queues / background processing. Ingestion is I/O- and compute-heavy, so /ingest enqueues work and returns immediately with a job summary. Locally this uses FastAPI BackgroundTasks; the same producer interface targets Celery + Redis/RabbitMQ for multi-worker scale-out.

Observability. Every request carries an id and emits structured JSON logs with route, latency_ms, retrieved chunk ids and (when using a hosted LLM) token usage. Langfuse tracing hooks activate automatically when LANGFUSE keys are present, giving per-step traces of the agentic flow.

Scalability path. The API is stateless, so it scales horizontally behind a load balancer. Vector store -> Astra DB / Chroma server; graph -> Neo4j Aura; cache + broker -> managed Redis; ingestion workers -> Celery pool. The provider abstraction lets us route cheap models for classification and stronger models for synthesis (LLM routing).

7. Key Decisions, Assumptions & Limitations

Decisions. Offline-first defaults (NumPy vector index, NetworkX graph, extractive fallback) were chosen so the prototype is trivially runnable and reproducible for grading, while every layer is interface-bound for a clean upgrade to managed services. LiteLLM gives provider independence and avoids lock-in.

Assumptions. Small corpus (single-digit to low-hundreds of documents); single-tenant internal use; documents are non-sensitive fictional samples; English content.

Limitations. Entity extraction is rule/heuristic-based (dictionary + patterns) rather than a fine-tuned NER model, so recall on novel entity names is limited. The NumPy index is in-process and not sharded — fine for the prototype, replaced by Astra/Chroma at scale. The extractive fallback answers are grounded but less fluent than a hosted LLM.

Future improvements. LLM-based NER + relation extraction, hybrid BM25+vector retrieval with cross-encoder reranking, an evaluation harness with RAGAS-style metrics wired into CI, streaming responses, a web UI, and full voice (Whisper STT + TTS).

Companion documents: README.md (setup & usage), SYSTEM_DESIGN.md (extended narrative), diagrams/ (source PNGs), and the running Swagger UI at /docs.